

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-платформы маркетплейса электронной техники
с системой управления заказами и аналитикой»

Выполнил: Нестерчук Антон Васильевич

Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Нестерчуку Антону Васильевичу,

обучающемуся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-платформы маркетплейса электронной техники с системой управления заказами и аналитикой»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	8	03.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	04.04.2026	
Задача 1.3. Формулировка цели и задач работы	7	06.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории и формирование требований к маркетплейсу	7	08.04.2026	
Задача 2.2. Описание функциональности платформы (UseCase, бизнес-логика ролей)	7	10.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ERD и схемы БД)	8	14.04.2026	
Задача 2.4. Разработка макетов страниц интерфейса пользователей платформы	5	18.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры клиентской части приложения	8	21.04.2026	
Задача 3.2. Разработка серверной части, реализация аутентификации и авторизации	8	27.04.2026	
Задача 3.3. Реализация CRUD-интерфейса для пользователей, товаров и заказов	9	03.05.2026	
Задача 3.4. Реализация системы корзины покупателя и бизнес-логики обработки заказов	9	09.05.2026	
Задача 3.5. Разработка модуля отчетности и работы с файлами	7	15.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	22.03.2026	
Задача 4.2. Деплой приложения на хостинг	4	26.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«30» марта 2026 г.

(подпись)

А. С. Кружалов

Дата выдачи задания

«30» марта 2026 г.

Дата сдачи выполненной работы

«26» мая 2026 г.

Задание принял к исполнению

«31» марта 2026 г.

(подпись)

А. В. Нестерчук

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих платформ электронной коммерции . . .	6
1.2 Анализ программных инструментов разработки	7
1.3 Формулировка цели и задач работы	9
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	10
2.1 Анализ целевой аудитории и формирование требований	10
2.2 Описание функциональности платформы	11
2.3 Проектирование модели данных	14
2.4 Разработка макетов страниц	17

ВВЕДЕНИЕ

Актуальность темы. Современный рынок розничной торговли электроникой характеризуется высоким уровнем конкуренции и постепенным переходом от классических интернет-магазинов к агрегаторам. Потребители все чаще предпочитают приобретать смартфоны, компьютерные комплектующие и периферию на платформах, объединяющих предложения от различных поставщиков, что позволяет сравнивать цены и выбирать оптимальные условия. Однако глобальные маркетплейсы зачастую перегружены товарами из других категорий, обладают сложной системой комиссий для продавцов и не предоставляют специализированных инструментов фильтрации, необходимых при выборе электронной техники.

В связи с этим возникает потребность в создании нишевых, профильных торговых платформ. Разработка специализированного маркетплейса электронной техники, предоставляющего малым и средним ритейлерам изолированные рабочие кабинеты, а администрации — прозрачные инструменты управления товарными потоками и аналитики, является актуальной задачей в сфере веб-разработки.

Проблематика. Существующие готовые системы управления контентом требуют значительных трудозатрат для адаптации под многопользовательскую архитектуру, где каждый продавец должен иметь независимый доступ к своим товарам и заказам без риска вмешательства в данные других участников. Использование современных веб-фреймворков позволяет решить эту проблему за счет проектирования гибкой ролевой модели, микросервисной архитектуры и оптимизированных реляционных баз данных.

Объектом исследования является организация процесса дистанционной торговли цифровой техникой в рамках многопользовательских платформ.

Предмет исследования — архитектурные подходы, алгоритмы агрегации данных и программные технологии, применяемые для разработки серверной и клиентской частей веб-маркетплейса.

Цель курсового проекта заключается в проектировании и программной реализации веб-платформы маркетплейса электронной техники, обеспечивающей взаимодействие продавцов и покупателей, а также сбор статистических данных для администратора.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих платформ электронной коммерции

Разработка платформы электронной коммерции по модели маркетплейса для продажи цифровой техники является сложной инженерной задачей. Специфика предметной области требует обработки глубоко вложенных характеристик товаров (для реализации системы сложных фильтров), обеспечения строгой изоляции данных независимых продавцов и высокой скорости отклика пользовательского интерфейса.

Анализ современного рынка готового программного обеспечения выявляет три основных подхода к созданию подобных платформ. Рассмотрим их технические особенности:

1. **Сборки на базе WordPress (WooCommerce).** Open Source⁽¹⁾ решение. Главный недостаток — использование потенциально большого количества атрибутов у сущностей в базе данных. При работе с большим каталогом электроники и множественными фильтрами производительность системы падает.
2. **Специализированные CMS (CS-Cart Multi-Vendor).** Коробочное решение с готовой логикой для маркетплейсов. Недостаток — устаревающая монолитная архитектура с серверным рендерингом, что приводит к постоянным перезагрузкам страниц и снижению отзывчивости интерфейса.
3. **Корпоративные решения (1C-Bitrix Enterprise).** Мощная платформа с интеграцией складского учета. Отличается высокой стоимостью лицензии и закрытостью ядра, что сильно усложняет и удорожает внедрение современных одностраничных приложений (SPA) на базе API.

Для структурирования полученных данных был проведен сравнительный анализ по ключевым критериям разработки. Результаты представлены в таблице 1.1.

(1) Распространяется под свободной лицензией GNU GPL.

Таблица 1.1 – Сравнительный анализ платформ электронной коммерции

Название	Тип архитектуры	Гибкость модели БД	Готовность к SPA/API
WordPress + WooCommerce	Монолит	Низкая	Низкая
CS-Cart Multi-Vendor	Монолит	Средняя	Средняя
1C-Bitrix Enterprise	Монолит + REST API	Высокая	Требует написания прослойки API

Детальный анализ существующих программных решений показал, что подавляющее большинство из них опирается на монолитную архитектуру и устаревшие подходы к рендерингу, что делает их медленными при работе с большими каталогами электроники. Использование готовых CMS накладывает ограничения на структуру базы данных, усложняя реализацию быстрых фильтров.

В связи с этим подтверждается целесообразность разработки собственного программного продукта. Разделение системы на независимый серверный API и клиентское приложение позволит добиться максимальной гибкости ролевой модели, высокой скорости отклика интерфейса и оптимизации работы с реляционной базой данных.

1.2 Анализ программных инструментов разработки

Для реализации проекта выбран современный стек технологий, обеспечивающий разделение приложения на независимые клиентскую и серверную части. Выбор инструментов определен требованиями к масштабируемости, скорости отклика интерфейса и безопасности данных:

- **Frontend-библиотека (React.js):** На современном рынке клиентской разработки доминируют такие фреймворки, как Angular, Vue.js и библиотека React. Angular обладает излишне жесткой архитектурой, которая избыточна для данного проекта, а Vue.js имеет менее обширную систему готовых инструментов для управления состояниями. Выбор библиотеки

React обусловлен компонентным подходом, использованием виртуальной древовидной структуры объектов, что обеспечивает высокую скорость перерисовки интерфейса каталога при применении множества фильтров, а также наличием официальных рекомендаций по разработке интерфейсов [1].

- **Стилизация (Tailwind CSS):** Для стилизации компонентов традиционно применяются CSS-препроцессоры или готовые UI-библиотеки, например, Bootstrap. Стоит отметить, что препроцессоры усложняют поддержку кода при росте проекта, а UI-библиотеки сильно ограничивают уникальность дизайна. Фреймворк Tailwind CSS был выбран для ускорения верстки интерфейса и минимизации объема CSS-кода за счет использования утилитарных классов, подробное описание которых приводится в документации [2].
- **Backend-платформа (Python + FastAPI):** При выборе серверной технологии рассматривались такие популярные решения, как Node.js и Python-фреймворки (Django, Flask). Node.js требует сторонних решений для строгой валидации данных, Django является тяжеловесным монолитом, а Flask не имеет встроенной асинхронности. В свою очередь, FastAPI обеспечивает высокую производительность за счет асинхронности и автоматическую генерацию документации с помощью Swagger, что необходимо для обработки запросов от множества пользователей одновременно [3].
- **СУБД (PostgreSQL):** Для хранения данных рассматривались нереляционные (MongoDB) и реляционные (MySQL, PostgreSQL) СУБД. Нереляционные базы не подходят для платформ электронной коммерции, так как не обеспечивают надежность финансовых транзакций. В сравнении с MySQL, выбор PostgreSQL аргументирован лучшей поддержкой сложных связей и наличием типа данных JSONB, что позволит эффективно хранить динамические характеристики электронной техники при сохранении высокой надежности реляционной структуры [4].

Для реализации дополнительного функционала клиентской части используется библиотека `react-router-dom`, обеспечивающая маршрутизацию страниц без перезагрузки браузера.

В серверной части приложения для взаимодействия с СУБД PostgreSQL применяется ORM-библиотека SQLAlchemy. Она позволяет абстрагироваться от прямых SQL-запросов, предотвращает уязвимости к SQL-инъекциям и эффективно работает с объектами базы данных на языке Python. Для обеспечения безопасности доступа пользователей используется протокол JSON Web Token. Его выбор обусловлен отдельной архитектурой веб-приложения: токены не требуют хранения сессий на сервере и позволяют безопасно передавать данные о ролях пользователей между независимыми клиентом и сервером.

1.3 Формулировка цели и задач работы

На основе проведенного анализа инструментов и выявленных проблем существующих маркетплейсов была сформулирована цель курсового проекта.

Цель работы — проектирование и разработка веб-платформы маркетплейса электронной техники, обеспечивающей взаимодействие продавцов и покупателей с системой аналитической отчетности.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ предметной области и обосновать выбор программных инструментов разработки;
2. Сформировать функциональные требования к платформе и описать бизнес-логику основных ролей пользователей;
3. Спроектировать модель данных и макеты пользовательских интерфейсов;
4. Разработать клиентскую часть приложения (карточки товаров, корзина и личные кабинеты);
5. Реализовать серверную часть с модулями аутентификации, управления каталогом и обработки заказов;
6. Выполнить развертывание разработанной системы на хостинге и подготовить отчетную документацию.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории и формирование требований

Анализ целевой аудитории позволяет выделить два основных сегмента пользователей. Первый сегмент — это розничные покупатели, заинтересованные в приобретении цифровой техники. Их главная потребность заключается в удобном поиске по специфическим техническим характеристикам и возможности сравнения предложений от разных поставщиков. Второй сегмент — представители малого и среднего бизнеса, нуждающиеся в доступном канале продажи своей продукции с минимальными техническими барьерами для входа.

Исходя из потребностей целевой аудитории, в проектируемой системе выделяются три ключевые ролевые модели с изолированными правами доступа:

- **Покупатель** — авторизованный пользователь, имеющий доступ к витрине товаров. Нуждается в инструментах фильтрации, управления корзиной заказов и отслеживания статусов текущих покупок через личный кабинет.
- **Продавец** — коммерческий пользователь, управляющий своими карточками товаров. Основные потребности продавца: публикация товаров с загрузкой медиафайлов, управление остатками, обработка поступивших заказов и получение агрегированной статистики продаж.
- **Администратор** — сотрудник платформы, обладающий полными правами. Отвечает за модерацию контента, блокировку нарушителей и управление глобальным справочником категорий техники.

На основе выделенных ролей был сформирован перечень различных требований к разрабатываемому программному продукту.

Функциональные требования:

1. Регистрация, аутентификация и авторизация пользователей с разграничением прав доступа на основе ролей.
2. Реализация CRUD-интерфейсов для управления профилями, каталогом товаров и категориями.

3. Наличие логики управления корзиной покупок и оформления заказа.
4. Маршрутизация заказов: продавец должен иметь доступ только к тем заказам, которые содержат добавленные им товары.
5. Возможность загрузки и хранения медиафайлов.
6. Выполнение агрегации данных на стороне сервера для формирования сводной аналитической отчетности продавца (подсчет выручки, количества заказов).

Нефункциональные требования:

1. Производительность: клиентская часть должна функционировать как одностраничное приложение (SPA), обеспечивая фильтрацию каталога без перезагрузки страницы.
2. Безопасность: пароли пользователей должны храниться в базе данных в виде криптографических хешей; передача сессионных данных должна осуществляться через подписанные JWT-токены.
3. Масштабируемость: структура базы данных должна быть нормализована для предотвращения аномалий при добавлении новых продавцов или характеристик товара.
4. Эргономичность: понятный интерфейс, удобная навигация и логичность расположения элементов в приложении.

Таким образом, проведенный анализ целевой аудитории и составление требований формируют четкий вектор для последующего моделирования бизнес-процессов и проектирования физической структуры реляционной базы данных.

2.2 Описание функциональности платформы

Для формализации функциональных требований и проектирования бизнес-логики разрабатываемой веб-платформы применена методология пользовательских историй. Данный подход позволяет описать функциональность программного продукта с позиции конечного потребителя. Рассмотрим ключевые сценарии взаимодействия для каждой роли в системе.

Сценарии для базовой роли **«Пользователь системы»**:

- *Как новый пользователь*, я хочу зарегистрироваться в системе, чтобы получить доступ к функционалу покупателя или продавца.
- *Как зарегистрированный пользователь*, я хочу авторизоваться в системе по своим учетным данным, чтобы безопасно управлять своим профилем.

Сценарии для роли «**Покупатель**»:

- *Как покупатель*, я хочу просматривать витрину и использовать фильтры, чтобы быстро находить нужную мне электронику.
- *Как покупатель*, я хочу переходить в карточку товара и добавлять его в корзину, чтобы формировать список покупок.
- *Как покупатель*, я хочу оформлять заказ из корзины и отслеживать его статус в личном кабинете, чтобы контролировать процесс доставки.

Сценарии для роли «**Продавец**»:

- *Как продавец*, я хочу иметь доступ к единой панели управления, чтобы централизованно вести коммерческую деятельность.
- *Как продавец*, я хочу управлять товарами и загружать к ним медиафайлы, чтобы качественно оформлять витрину.
- *Как продавец*, я хочу изменять статусы заказов и просматривать статистику продаж с возможностью выгрузки отчета в формате CSV, чтобы анализировать выручку.

Сценарии для роли «**Администратор**»:

- *Как администратор*, я хочу управлять категориями техники, чтобы поддерживать структурированность каталога.
- *Как администратор*, я хочу модерировать учетные записи пользователей, чтобы пресекать нарушения правил платформы.

На основе сформированных функциональных требований и пользовательских сценариев была разработана UML-диаграмма вариантов использования

(рисунок 2.1). Данная модель визуализирует границы маркетплейса и детализирует архитектуру взаимодействия ролей с использованием механизмов наследования и расширенной нотации связей.

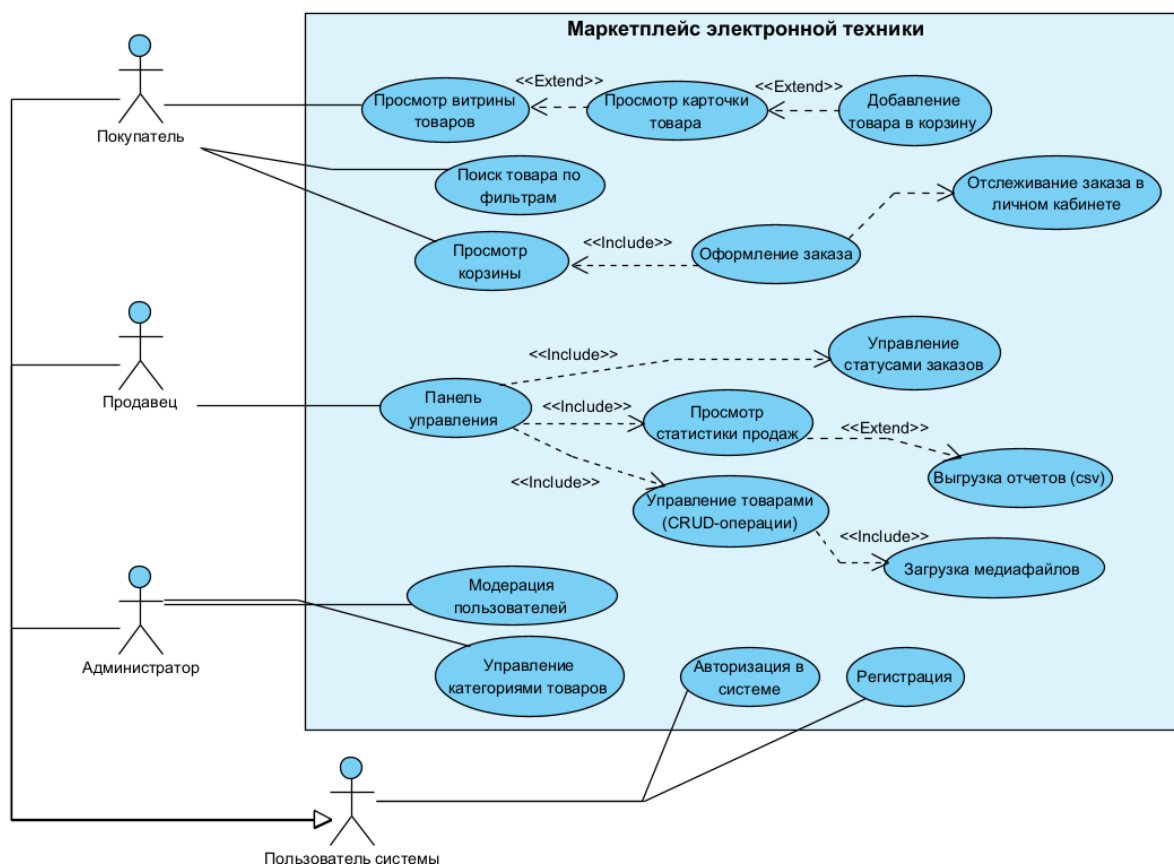


Рисунок 2.1 – Диаграмма вариантов использования (Use Case)

В системе реализована иерархия акторов: базовая абстракция «Пользователь системы» берет на себя общие прецеденты авторизации и регистрации, а остальные роли наследуют эти права. Бизнес-логика изобилует сложными зависимостями. Например, панель управления у продавцов включает в себя модули управления заказами, товарами и просмотра статистики. В свою очередь, создание карточки товара обязательно требует загрузки медиафайлов. Отношения расширения применены для дополнительного функционала, который вызывается опционально: выгрузка CSV-отчета из статистики или отслеживание заказа после его оформления. Разработанная поведенческая модель послужит надежным фундаментом для следующего этапа — проектирования физической структуры реляционной базы данных.

2.3 Проектирование модели данных

При проектировании инфологической модели были выделены следующие ключевые сущности:

- `roles` и `users` — таблицы для реализации системы авторизации и управления правами доступа.
- `categories` и `products` — таблицы для формирования структуры каталога и хранения карточек товаров. Каждая карточка товара жестко привязана к продавцу и категории.
- `orders` и `order_items` — таблицы для реализации бизнес-логики покупок.

Особое внимание при проектировании было уделено архитектурной проблеме заказов: в классическом интернет-магазине статус присваивается всему заказу целиком, однако в маркетплейсе один заказ может содержать товары от разных продавцов. Для решения этой проблемы статус заказа был вынесен на уровень позиций заказа. Это позволяет каждому продавцу изменять статус доставки только своего товара, не влияя на общую корзину покупателя.

Логическая структура базы данных со связями типа «один-ко-многим» представлена на ER-диаграмме (рисунок 2.2).

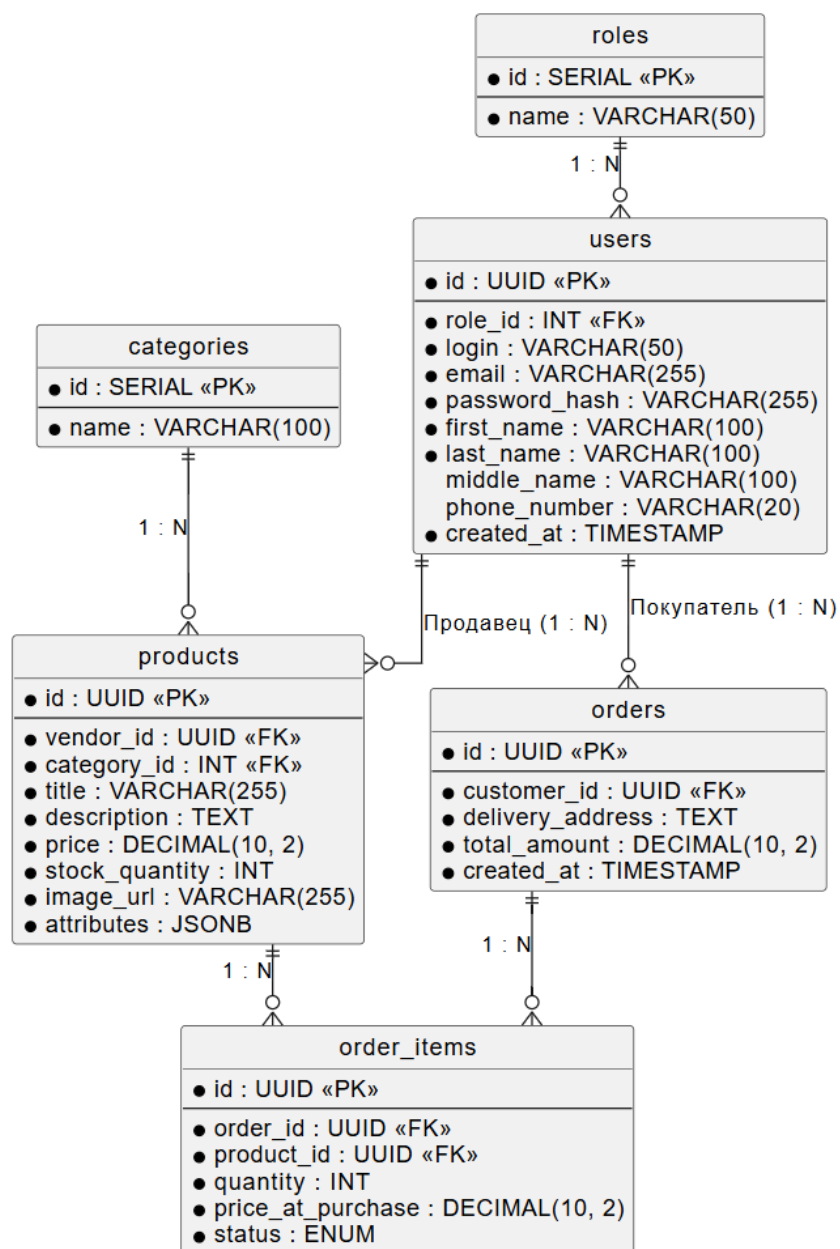


Рисунок 2.2 – ER-диаграмма базы данных

Для программной реализации спроектированная модель была нормализована до 3-й нормальной формы. Ниже (таблицы 2.1, 2.2, 2.3, 2.4) представлены физические схемы основных таблиц базы данных.

Таблица 2.1 – Структура таблицы пользователей (users)

Поле	Тип данных	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
role_id	INT	FOREIGN KEY	Ссылка на роль пользователя
login	VARCHAR(50)	UNIQUE, NOT NULL	Логин для входа

Продолжение таблицы 2.1

Поле	Тип данных	Ограничение	Описание
email	VARCHAR(255)	UNIQUE, NOT NULL	Электронная почта
password_hash	VARCHAR(255)	NOT NULL	Хеш пароля (bcrypt)
first_name	VARCHAR(100)	NOT NULL	Имя
last_name	VARCHAR(100)	NOT NULL	Фамилия
middle_name	VARCHAR(100)	NULL	Отчество (необязательно)
phone_number	VARCHAR(20)	NULL	Телефон для связи
created_at	TIMESTAMP	DEFAULT NOW()	Дата регистрации

Таблица 2.2 – Структура таблицы товаров (products)

Поле	Тип данных	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
vendor_id	UUID	FOREIGN KEY	Ссылка на продавца
category_id	INT	FOREIGN KEY	Ссылка на категорию
title	VARCHAR(255)	NOT NULL	Наименование товара
description	TEXT	NULL	Подробное описание
price	DECIMAL(10,2)	CHECK > 0	Стоимость товара
stock_quantity	INT	DEFAULT 0	Остаток на складе
image_url	VARCHAR(255)	NULL	Путь к файлу изображения
attributes	JSONB	NOT NULL	Динамические характеристики

Таблица 2.3 – Структура таблицы заказов (orders)

Поле	Тип данных	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
customer_id	UUID	FOREIGN KEY	Ссылка на покупателя
delivery_address	TEXT	NOT NULL	Полный адрес доставки
total_amount	DECIMAL(10,2)	NOT NULL	Итоговая сумма заказа
created_at	TIMESTAMP	DEFAULT NOW()	Дата и время оформления

Таблица 2.4 – Структура таблицы позиций заказа (order_items)

Поле	Тип данных	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
order_id	UUID	FOREIGN KEY	Ссылка на общий чек (заказ)
product_id	UUID	FOREIGN KEY	Ссылка на купленный товар
quantity	INT	CHECK > 0	Количество единиц товара
price_at_purchase	DECIMAL(10,2)	NOT NULL	Цена на момент покупки
status	ENUM	NOT NULL	Статус отправки продавцом

Спроектированная реляционная модель исключает дублирование данных и полностью готова к интеграции с серверной частью приложения через ORM SQLAlchemy.

2.4 Разработка макетов страниц

Для визуализации пользовательского интерфейса и проектирования путей взаимодействия были разработаны низкодетализированные структурные макеты основных страниц (Wireframes). Данный подход позволяет сфокусироваться на эргономике, расположении элементов управления и информационной архитектуре до начала этапа программной верстки. Проектирование макетов осуществлялось с учетом компонентного подхода, который будет реализован в дальнейшем.

Главная страница является точкой входа для покупателей. В верхней части расположена глобальная панель навигации с модулем глобального поиска, кнопками авторизации и доступа к корзине. Ниже расположен рекламный блок с призывом к действию (СТА). Ключевым элементом интерфейса является панель фильтрации и блок категорий, позволяющие покупателю отсеивать технику по заданным критериям. Основное пространство занимает модульная сетка популярных товаров. Макет главной страницы представлен на рисунке 2.3.

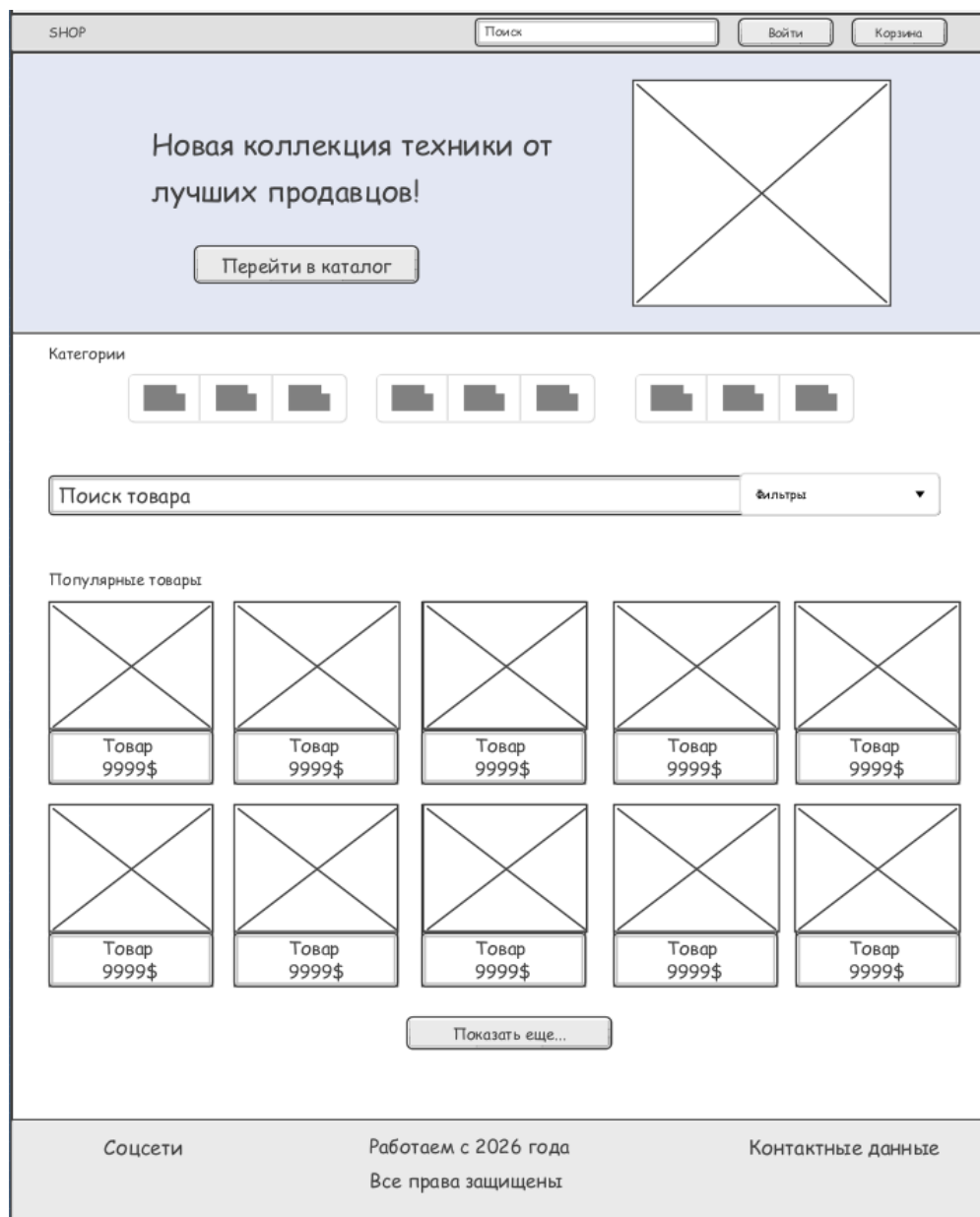


Рисунок 2.3 – Макет главной страницы

Карточка товара разработана с акцентом на детальное представление сложной электроники. Макет (рисунок 2.4) включает крупную область для просмотра фотографий товара. В информационной панели, помимо цены и названия, предусмотрен вывод имени продавца и динамических селекторов характеристик. Ниже располагаются блоки с текстовым описанием, техническими спецификациями и рекомендательной системой товаров из той же категории.



Рисунок 2.4 – Макет карточки товара

Интерфейс корзины (рисунок 2.5) спроектирован для удобной агрегации товаров от различных продавцов. Пользователь имеет возможность управлять количеством позиций и удалять неактуальные товары. Правая боковая панель зафиксирована и отображает информацию о заказе с кнопкой перехода к оформлению заказа.

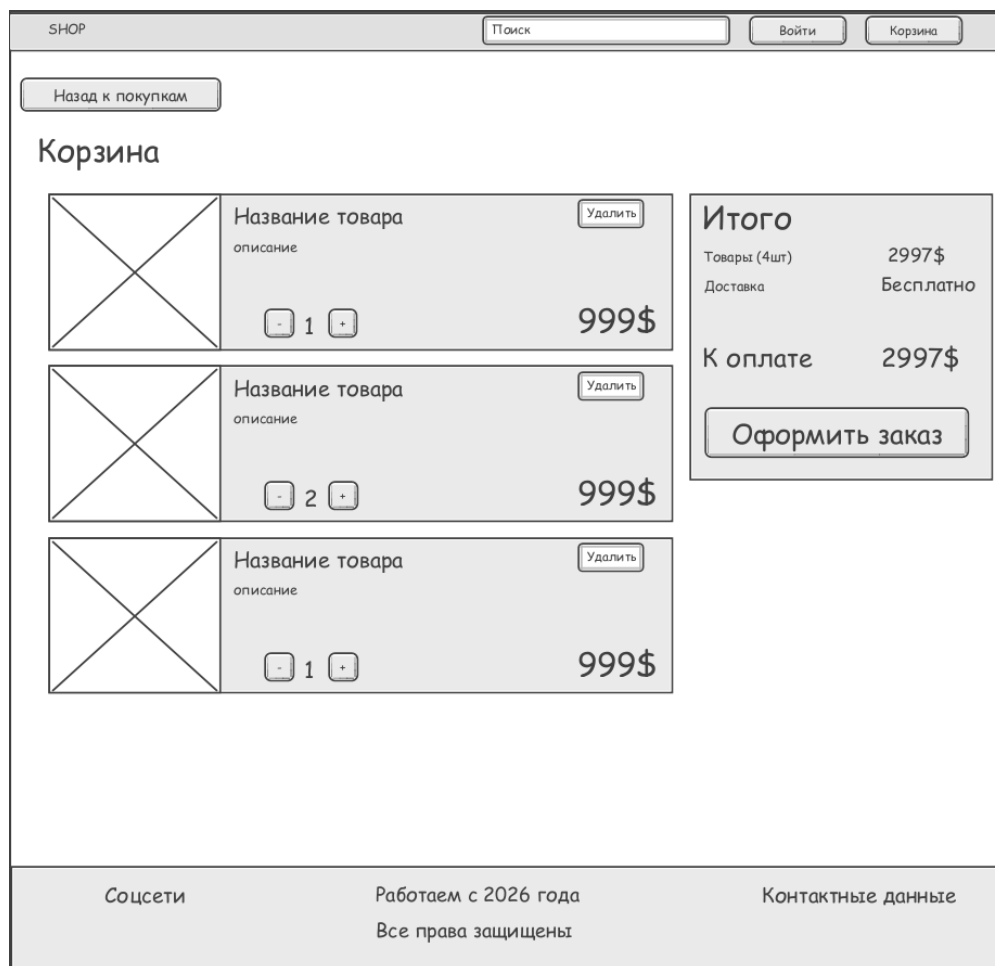


Рисунок 2.5 – Макет корзины

Для продавцов спроектирован изолированный интерфейс панели управления (рисунок 2.6). Макет имеет классическую для B2B-систем структуру, соответствующую современным паттернам проектирования коммерческих интерфейсов [5]: левое боковое меню для навигации между разделами и широкую рабочую область. В ней визуализированы графики агрегированной статистики продаж с функцией экспорта данных в формате CSV. Ниже расположены списки для выполнения CRUD-операций над карточками товаров и управления статусами поступивших заказов.

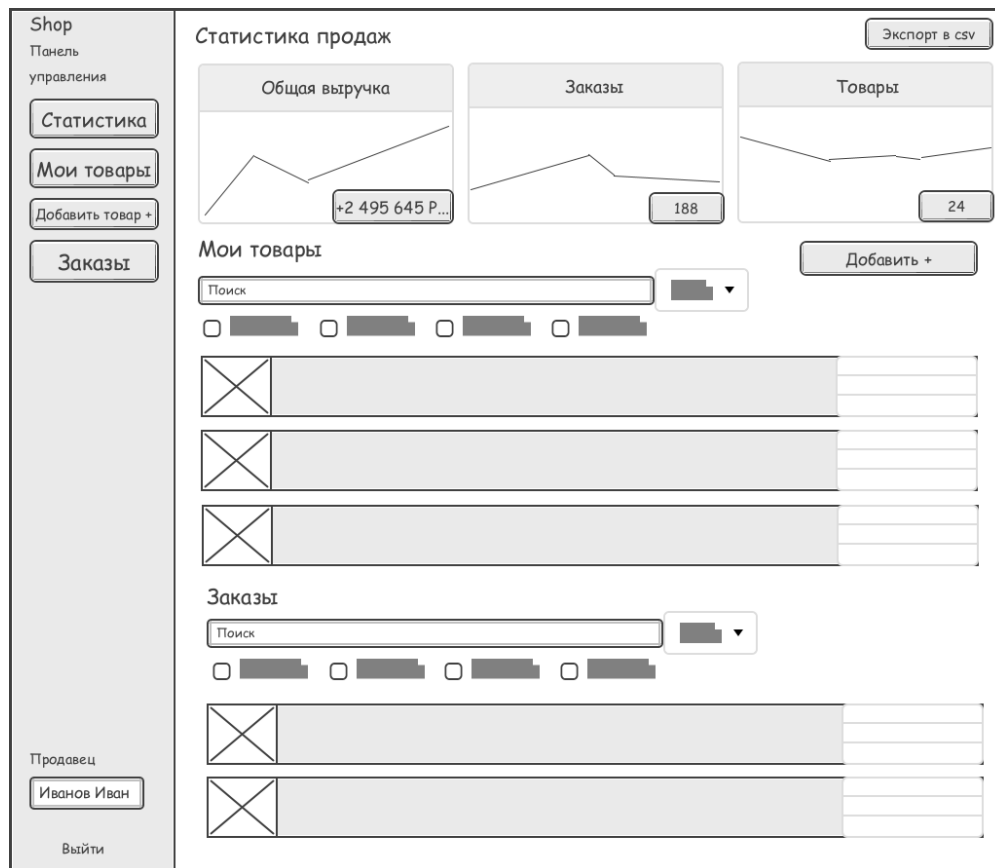


Рисунок 2.6 – Макет панели управления продавца

Таким образом, завершен этап архитектурного планирования, и подготовлена исчерпывающая база для программной реализации серверной и клиентской частей маркетплейса в следующей главе.

СПИСОК ЛИТЕРАТУРЫ

- [1] React Documentation: Quick Start [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://react.dev> (дата обращения: 29.04.2026).
- [2] Tailwind CSS Documentation: Utility-First Fundamentals [Electronic resource]. — [S. l. : s. n.]. — URL: <https://tailwindcss.com/docs> (online; accessed: 29.04.2026).
- [3] FastAPI Documentation: Professional API development [Electronic resource]. — [S. l. : s. n.]. — URL: <https://fastapi.tiangolo.com> (online; accessed: 29.04.2026).
- [4] PostgreSQL Documentation: JSON Types [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.postgresql.org/docs/current/datatype-json.html> (online; accessed: 29.04.2026).
- [5] Проектирование B2B-интерфейсов: паттерны, дашборды и аналитика [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://uxpub.ru/b2b-dashboard-design> (дата обращения: 04.05.2026).